

Neural network serialization formats: a pure-Go integration map for LOOM

Pure-Go (no-CGO) parsing of neural network weights is now feasible across more of the ecosystem than it was even 18 months ago, but the practical interop surface for LOOM collapses to roughly three formats that matter: safetensors, GGUF, and ONNX, with HDF5/pickle/SSTable being the historical blockers and BitNet TQ1_0/TQ2_0 dequant being LOOM's most strategically distinctive win. Of the ~20 formats surveyed, only safetensors, .npy/.npz, Darknet .weights, and llamafile are *trivial* in pure Go; GGUF parsing is well-documented but no pure-Go library dequantizes K-quants/I-quants/TQ-quants today; ONNX and the protobuf family are mechanical; PyTorch pickle and Keras HDF5 — historically the two hard blockers — are now solvable with caveats via `nlpodyssey/gopickle` and `scigolib/hdf5` respectively; TensorFlow SavedModel's SSTable variable index is the one genuinely unsolved blocker remaining; and TensorRT/CoreML execution are framework-locked and out of scope. The integration roadmap that maximizes ecosystem coverage per engineer-week is: **finish safetensors + write support + sharded index + FP8 dtypes → add GGUF reader with TQ2_0/TQ1_0 then Q8_0/Q4_K_M dequant → add ONNX protobuf parsing for an LLM-shaped operator subset → add NumPy + Flax msgpack → add Darknet + llamafile → defer Keras/HDF5, PyTorch pickle, TF SavedModel behind external conversion.**

This report dives into each format's container, encoding, versioning, and pure-Go feasibility, building toward a concrete priority order.

The interop landscape in one paragraph

Three formats serve as **interchange hubs**: `safetensors` (HuggingFace's tensor-only container, now the de-facto distribution format for HF Transformers since 2023), `ONNX` (the protobuf-based cross-framework graph standard), and `GGUF` (llama.cpp's local-inference standard). Everything else either feeds into these hubs (PyTorch → ONNX, HF → GGUF, Keras → SavedModel → ONNX) or is a **runtime-locked compilation target** consumed by exactly one inference engine: `TensorRT .engine` (NVIDIA, GPU-architecture-specific, undocumented), `CoreML .mlpackage` (Apple Neural Engine), `OpenVINO IR` (Intel), `TFLite` (Google's mobile interpreter, schema is public but op kernels are the moat), `ExecuTorch .pte` (PyTorch Edge). LOOM's reach is maximized by reading from the three hubs and refusing the runtime-locked targets with a clear conversion message.

safetensors: trivially the right primary format

The format is **8 bytes little-endian u64 header length**, then **UTF-8 JSON header**, then **raw tensor bytes**. The JSON header maps tensor name → `{dtype, shape, data_offsets: [start, end]}` with offsets relative to the start of the data region; an optional `__metadata__: Dict[str, str]` key holds free-form metadata. There is **no magic number, no version field, and no v2 format** — the on-disk format has been frozen since 2022-09. Compatibility evolves only by extending the dtype enum: `BOOL`, `U8/I8`, `F8_E5M2`, `F8_E4M3`, `F8_E5M2FNUZ`, `F8_E4M3FNUZ`, `I16/U16`, `F16`, `BF16`, `I32/U32`, `F32`, `F64`, `I64/U64`, `C64`, and the newly added packed sub-byte types like `float4_e2m1fn_x2`. Hard limits: 100 MB header cap; tensor data regions **MUST NOT** overlap (a validating reader can rely on this for safe mmap). The format guarantees no arbitrary code execution and supports zero-copy mmap reads. [GitHub + 13](#)

Sharding (models > 5 GB) uses the convention `model-NNNNN-of-MMMMM.safetensors` files plus a `model.safetensors.index.json` with shape `{"metadata": {"total_size": N}, "weight_map": {"tensor_name": filename}}`. Each shard is an independently valid safetensors file. [Hugging Face](#)

Quantization is out-of-band: safetensors stores only raw tensors. Quantized models (GPTQ, AWQ, bitsandbytes, FP8, compressed-tensors) encode quantization as **additional tensors with naming conventions** plus a `quantization_config` block in the companion `config.json`. Typical GPTQ layout per linear: `qweight` (I32-packed 4-bit codes), `qzeros`, `scales` (F16), `g_idx`. FP8 layout: `weight: F8_E4M3` + sidecar `weight_scale: F32` + `input_scale: F32`. A LOOM safetensors loader must pair tensors with their scale/zero-point siblings per architecture convention — there is no universal contract. [vLLM](#)

Pure-Go state: github.com/nlpodyssey/safetensors is the recommended baseline — pure Go, read + write + streaming `SerializeToWriter`, packaged in Debian. github.com/sugarme/safetensors exists but is older and read-focused. Implementation cost for LOOM to fill in writer + sharded reader + FP8 dtypes is **1-2 days at most**. This is essentially a solved problem.

[Go Packages](#)

GGUF and the ggml family: the format that matters most for LOOM

GGUF is llama.cpp's tensor + metadata container, **little-endian throughout**, with magic bytes `'G', 'G', 'U', 'F'` (read as little-endian `uint32 = 0x46554747`). Version history: [DeepWiki](#)

Format	Magic	Notes
GGML	0x67676d6c	Pre-March 2023, unversioned, no alignment GitHub GitHub
GGMF v1	0x67676d66	Adds embedded vocab + version field GitHub GitHub
GGJT v1/ v2/v3	0x67676a74	mmap alignment; quant version encoded in <code>f_{type}/1000</code> ; the v1→v2 and v2→v3 transitions changed quantization schemes GitHub githubusercontent
GGUF v1	0x46554747	Reworked metadata as key-value, PR #302 merged Aug 2023
GGUF v2	same magic, version=2	String/array/dimension counts promoted from u32 to u64 githubusercontent
GGUF v3	same magic, version=3	Big-endian support added; githubusercontent current spec

File layout (v3): `[magic u32 | version u32 | tensor_count u64 | metadata_kv_count u64 | metadata_kv[N] | tensor_infos[N] | padding | tensor_data]`. Tensor data is aligned to **32 bytes by default** (overridable via the `general.alignment` metadata key). Each tensor info entry is `{name: gguf_string (u64 len + utf8), n_dims: u32, dimensions: u64[n_dims], type: u32, offset: u64}` where offset is relative to the start of the tensor data region and must be a multiple of ALIGNMENT. [DeepWiki + 4](#)

Metadata KV values use a typed enum: UINT8/16/32/64, INT8/16/32/64, FLOAT32/64, BOOL, STRING (`(u64 len + utf8 bytes)`), and ARRAY (recursive — `elem_type u32 + len u64 + values`). Well-known keys follow a dotted lowercase namespace: required keys are `general.architecture` (e.g. `llama`, `qwen2`, `qwen3`, `mistral`, `mixtral`, `phi3`, `gemma`, `bitnet`, `bitnet-b1.58`), `general.quantization_version`, `general.alignment`. Per-architecture keys use the arch prefix: `llama.context_length`, `llama.embedding_length`, `llama.block_count`, `llama.attention.head_count`, `llama.attention.head_count_kv`, `llama.rope.dimension_count`, `llama.rope.freq_base`, `llama.attention.layer_norm_rms_epsilon`. Tokenizers embed via `tokenizer.ggml.{model, tokens, :antCitation[] {citations="683d0f04-1c93-4655-81f0-23d63d46350c" injected="space"} scores, token_type, merges, bos_token_id, eos_token_id, ...}}` and increasingly `tokenizer.chat_template` (Jinja). [githubusercontent + 5](#)

The quant zoo and what to implement

The `ggml_type` enum spans F32, F16, BF16, all the legacy block quants (Q4_0/1, Q5_0/1, Q8_0/1 — 32-element blocks with fp16 scale $\pm$ min), the **K-quants** (Q2_K through Q8_K — 256-element super-blocks with sub-block scales, the workhorse for modern LLM quant), the **I-quants** (IQ1_S, IQ1_M, IQ2_XXS/XS/S, IQ3_XXS/S, IQ4_NL/XS — codebook-based, with grids that **must be transcribed verbatim** from `ggml-quants.c`), and the new entries that matter most for LOOM:

[DeepWiki](#)

TQ1_0 (1.6875 bpw) and TQ2_0 (2.0625 bpw) were added in `ggml-org/llama.cpp` PR #8151 (compilade, merged August 2024) specifically for BitNet b1.58 and TriLM ternary models. Block size is 256 in both cases. **TQ2_0** is straightforward — 4 trits per byte at stride 32, packed as $((xi \& 3) \ll (2*n))$ where $xi \in \{0,1,2\}$ maps to $\{-1, 0, +1\}$, with a single fp16 scale per block; dequant is $((qs \gg (2*n)) \& 3 - 1) * d$. **TQ1_0** is the clever one: it packs 5 trits per byte using a base-3 encoding scaled into a byte by ceiling-divide-by-243, so successive trits can be extracted by repeated `byte * 3` operations where the MSD comes out of the upper byte. The first 240 weights of each 256-block go into `qs[0..47]` with 5 trits/byte, the remaining 16 weights go into `qh[0..3]` with 4 trits/byte using the simpler TQ2_0-style 2-bit packing. [arXiv + 5](#)

The catch: block size 256 means any tensor row not divisible by 256 falls back to F16 — Microsoft's BitNet 1.3B with FFN=5460 dimensions can't fully ternarize, ending up at ~5.01 effective bpw. Output and embedding tensors are also typically kept in Q4_K/Q6_K rather than ternary. [arxiv + 3](#)

Pure-Go GGUF libraries:

- github.com/gpustack/gguf-parser-go — the canonical pure-Go parser, used by GPUStack in production, supports remote chunked HTTP reads from HuggingFace, identifies all quant types — but **does not dequantize tensor data**. [GitHub](#)
- github.com/abrandner/gguf — minimal pure-Go reader for v1/v2/v3 headers and raw tensor byte access. [GitHub](#)
- github.com/ollama/ollama `fs/ggml` — pure Go for metadata, but Ollama executes via CGO into llama.cpp; tightly coupled to ollama internals.
- **No pure-Go library today dequantizes K-quants, I-quants, or TQ-quants.** This is the green-field opportunity.

Effort estimate: GGUF v1/v2/v3 reader + writer = ~1 week. F32/F16/BF16 access + legacy block quants (Q4_0, Q4_1, Q5_0, Q5_1, Q8_0) = ~1-2 days. K-quants (Q4_K, Q5_K, Q6_K with their packed 6+4-bit scales) = 4-6 days. I-quants (with codebook tables transcribed) = 5-8 days. **TQ2_0 = 0.5 day; TQ1_0 = 1-2 days.** Full coverage matching llama.cpp = 2-3 months. **Total to "load any modern HF safetensors model + any BitNet/TriLM GGUF": ~3-4 weeks of focused work.**

The authoritative spec lives at `ggml/docs/gguf.md` in `ggml-org/ggml`; block struct definitions in `ggml/src/ggml-common.h`; reference dequant code in `ggml/src/ggml-quants.c`.

ONNX: the universal interop graph, mechanical in pure Go to parse

ONNX is **protobuf** (deliberately using only the proto2/proto3 intersection so wire format is identical) defined in `github.com/onnx/onnx/blob/main/onnx/onnx.proto`. Top hierarchy: `ModelProto { ir_version, opset_import[], graph, functions, training_info, metadata_props } → GraphProto { node[], initializer[], sparse_initializer[], input/output/value_info, quantization_annotation[] } → NodeProto { op_type, domain, input[], output[], attribute[] }`. Tensors stored either as packed typed fields, as `raw_data` (little-endian bytes — required for sub-byte types which are LSB-first packed 2-per-byte or 4-per-byte), or via **external data** (when the protobuf 2 GB limit is hit) referencing a sibling `.bin` file by offset+length, with offsets ideally aligned to 4096 bytes for mmap. `github + 3`

IR and opset versioning

IR versions advance with major schema changes. From IR 1 (2017-10) through IR 13 (2025-11): IR 3 added operator versioning, IR 4 added BFLOAT16, IR 5 added quantization annotations, IR 6 added sparse tensors, IR 7 added training info, IR 9 added FP8 (FLOAT8E4M3FN, E4M3FNUZ, E5M2, E5M2FNUZ), IR 10 added INT4/UINT4 plus `metadata_props` everywhere, IR 11 added FLOAT4E2M1 and multi-device sharding, IR 12 added FLOAT8E8M0 (the OCP MX scale type), IR 13 added INT2/UINT2. `GitHub` `github`

Opset versions (`ai.onnx` domain) currently run through opset 23-25 depending on tool. The strategic milestones for LLM inference:

- **Opset 10** (2019): initial QuantizeLinear/DequantizeLinear and MatMulInteger
- **Opset 17** (2022): LayerNormalization standardized `New Releases`
- **Opset 19** (2023): FP8 in Q/DQ
- **Opset 21** (2024): **INT4/UINT4 types**, block quantization in Q/DQ — required for AWQ/GPTQ `ONNX Runtime`
- **Opset 23** (2025): **Attention, RotaryEmbedding, RMSNormalization** standardized

The auxiliary `ai.onnx.ml` opset is at v5 for classical ML; `ai.onnx.training` is effectively dormant.

com.microsoft contrib ops — what production LLM ONNX actually uses

99% of HuggingFace-exported LLM ONNX models in 2025–2026 still rely on `com.microsoft` extensions because they predate or supplement the standard Attention-23. The critical ones to recognize: `MatMulNBits` (the workhorse for 4-bit weight-only quant — packed N-bit weights with `block_size`, separate fp16 scales and optional `zero_points`), `GroupQueryAttention` (KV cache with `past_key`/`past_value`/`seq_lens_k` inputs), `MultiHeadAttention`, `RotaryEmbedding`, `SkipSimplifiedLayerNormalization`, `SimplifiedLayerNormalization` (~RMSNorm), `FastGelu`, `BiasGelu`, `MatMulBnb4` (bitsandbytes NF4), `GatherBlockQuantized` (quantized embedding lookup), `QMoE`, `CausalConvWithState` (Mamba/SSM), and the generation ops `BeamSearch`/`GreedySearch`/`SamplingOp`. [GitHub + 3](#)

The .ort sister format

`.ort` is the ONNX Runtime FlatBuffers-based mobile format. Schema at `microsoft/onnxruntime/onnxruntime/core/flatbuffers/schema/ort.fbs`, file identifier `"ORTM"`. The schema mirrors the protobuf model as FB tables, but graph optimizations are pre-applied and kernel type strings pre-resolved, enabling zero-copy weight access via `session.use_ort_model_bytes_directly = 1` and minimal-build binaries under 2 MB. No `external_data` path — weights inline, 2 GB limit applies. ORT version is decoupled from onnxruntime version.

Pure-Go state

- github.com/AdvancedClimateSystems/gonnx — best pure-Go reference, custom tensor impl based on gorgonia.org/tensor, targets **opset 13**. Active. Covers common conv/matmul/gemm/pool/reshape ops; **no Attention, no RoPE, no FP8, no 4-bit, no external_data**. [GitHub + 2](#)
- github.com/oramasearch/onnx-go (revived from archived [owulveryck/onnx-go](https://github.com/owulveryck/onnx-go)) — parsing + Gorgonia backend covering ~25% of ops. Experimental. [GitHub + 3](#)
- github.com/gomlx/onnx-gomlx — excellent pure-Go protobuf parsing and round-trip; executor depends on GoMLX/XLA (typically not pure-Go in practice).
- github.com/yalue/onnxruntime_go and the purego variants — bind to libonnxruntime dynamically; not pure-Go in spirit. [Go Packages](#)
- No mature pure-Go `.ort` parser exists, but `flatc --go ort.fbs` produces working bindings in a weekend.

Effort estimate: parse/write `.onnx` (any IR 1-13) = **1-2 weeks** by regenerating Go bindings from `onnx.proto`. External-data resolver with mmap = 0.5 week. Sub-byte unpacking helpers + FP8/FP4 conversion routines = 0.5 week. Parse-only `.ort` = 1 week. **Execution engine** for a curated LLM subset (Attention-23, RoPE-23, RMSNorm-23, LayerNorm-17, MatMul, MatMulNBits, Gather, Add/Mul/Softmax/SiLU, Reshape/Transpose/Concat, Q/DQ-INT4, GQA + KV cache) = **8-16 weeks**. Full coverage of all 190+ ops × version history = **50-150 engineer-weeks** — effectively rewriting ORT, not advised.

PyTorch: a ZIP of pickle + raw storages, dangerous but tractable for state_dicts

A modern `torch.save()` file (since PyTorch 1.6) is a **ZIP archive** containing `data.pkl` (the pickle stream describing the Python object graph, default protocol 2 → starts with `\x80\x02`), `data/N` (raw tensor storage files, one per storage), a `version` text file, a `byteorder` text file (added ~PyTorch 2.1), and `.data/serialization_id`. The pickle uses a **persistent_id** mechanism: every `torch.Storage` is referenced as `("storage", StorageClass, key, location, numel)`, with the integer `key` mapping to a file inside `data/`. Tensors are reconstructed via REDUCE calls to `torch._utils._rebuild_tensor_v2(storage, storage_offset, size, stride, requires_grad, backward_hooks)` — view aliasing (multiple tensors sharing one storage) falls out of this naturally. Storage records are 64-byte aligned for mmap.

Pickle's danger is structural: opcodes `GLOBAL`, `STACK_GLOBAL`, `REDUCE`, `BUILD` resolve to arbitrary Python callable invocation, and a malicious `__reduce__` can run code at load time.

PyTorch 2.6 (Jan 2025) flipped `weights_only=True` to default: `torch/_weights_only_unpickler.py` reimplements the pickle state machine and refuses `GLOBAL` lookups not in an allowlist (`torch._utils._rebuild_*`, `torch.{FloatStorage, HalfStorage, BFloat16Storage, ...}`, `collections.{OrderedDict, Counter}`, `torch.nn.Parameter`). A pure-Go loader following the same allowlist is **safer by construction**. [GitHub](#) [GitHub](#)

The legacy pre-1.6 non-zip format is five concatenated pickle streams followed by raw tensor blobs in storage-key order; still appears in older Detectron/fastai artifacts.

TorchScript archives (`torch.jit.save`) share the `.pt` extension but contain `code/__torch__.py` (TorchScript source) + `constants.pkl` + the same `data/` directory.

Distinguishing heuristic: TorchScript archives have a `code/` directory; eager `torch.save` archives don't. Weights are extractable; bytecode execution is out of scope. [GitHub](#) [MONAI](#)

The newer `.pt2` / `torch.export` `ExportedProgram` (PyTorch 2.x) is also a ZIP but stores the graph as **JSON** (driven by `torch/_export/serde/schema.py` — proto-versioned `@dataclass` definitions), with `data/weights/` raw storages, optional `data/aotinductor/<model>-<backend>/` precompiled .so libraries, and `data/sample_inputs/`. Operator references are `torch.ops.aten.*` strings; the schema is unstable and `SCHEMA_VERSION` churns frequently.

Pure-Go pickle state

- `github.com/kisielk/og-rek` (**ogórek**) — pure Go, protocols 0–5 mostly, fuzz-tested, exposes `PersistentLoad` callback (exactly what PyTorch needs), decoded GLOBAL/REDUCE produce inert `Class`/`Call` values rather than executing — safe-by-default. Caveat: pickle **protocol 5** support is incomplete (issue #62 open); modern PyTorch defaults to protocol 2 but some research code uses 5 via `HIGHEST_PROTOCOL`.
- `github.com/nlpodyssey/gopickle` — purpose-built for PyTorch loading, has a `pytorch` subpackage with built-in handling of `_rebuild_tensor_v2`, `OrderedDict`, `Storage` classes. Supports modern ZIP and legacy non-tar formats; **does NOT support TorchScript or legacy-tar formats**; self-described as alpha. Real-world issue: pandas-style `.pt` files with `numpy.core.multiarray._reconstruct` REDUCE calls fail without user-supplied callable registrations. **Best starting point for LOOM's PyTorch path.** [GitHub](#)
- `github.com/hydrogen18/stalecucumber` — older, limited to protocols 0–2, no `persistent_id` hook suitable for PyTorch.

Realistic LOOM strategy

Target `state_dict` files (HF/timm/torchvision distributions): open as ZIP, find `data.pkl`, decode pickle with `ogórek` (or `nlpodyssey/gopickle`), install a `persistent_load` that intercepts storage tuples and returns sentinel structs recording (key, dtype, numel), walk the resulting `Call` tree expecting `_rebuild_tensor_v2(storage, offset, size, stride, ...)`, open `data/<key>` from the zip, slice and reinterpret bytes per dtype. ~**1 engineer-week** for f32/f16/bf16/i8/i16/i32/i64/u8/bool/complex64 dtypes with view aliasing and the legacy format as a secondary path. **Arbitrary pickled `nn.Module` subclasses are open-ended and brittle** — recommend rejection. **TorchScript and `.pt2` ATen-IR execution are out of scope** — extract weights only.

TensorFlow and Keras: the SSTable and HDF5 questions

TensorFlow SavedModel

A directory with `saved_model.pb` (a `SavedModel` proto containing `MetaGraphDef[]` containing `GraphDef`), `variables/variables.index` + `variables.data-NNNNN-of-MMMM` shards, `fingerprint.pb` (TF 2.11+), and optional `assets/`. **Frozen `.pb` graphs from TF1** embed weights directly in `Const` `NodeDef.AttrValue.TensorProto` — parseable with **only protobuf**, no SSTable, no HDF5. This is the easy TF case (~2 days).

The hard case is the **TF checkpoint v2 bundle**: `variables.index` is a **LevelDB SSTable** (Google's Bigtable-style immutable sorted string table) where keys are tensor names like `"layer_1/kernel/.ATTRIBUTES/VARIABLE_VALUE"` and values are serialized `BundleEntryProto { dtype, shape, shard_id, offset, size, crc32c }`. Reading requires implementing: footer parsing at end of file (TF's tweaked magic `0x57e3e1eebb9d5b1c`), varint-encoded `BlockHandles`, prefix-compressed key encoding with restart points every 16 entries, optional Snappy block decompression (rare in checkpoints), CRC32C Castagnoli validation (covered by Go's `stdlib hash/crc32`), then random-access reads of `variables.data-NNNNN-of-MMMM` shards by offset. [GitHub + 3](#)

No pure-Go TF checkpoint v2 reader exists. `tensorflow/go`, `galeone/tfgo`, and `gomlx/onnx-gomlx`'s TF paths all use CGO. [github.com/wamuir/graft](#) provides TF protobuf Go bindings without CGO (useful as a stub dep), but no checkpoint reader. Porting LevelDB's `table/` directory (~800 lines of C++) to Go is finite work: budget **2-3 weeks for a read-only SSTable port**, then **2 days for BundleEntryProto + shard reader**. [github.com/syndtr/goleveldb](#) has a pure-Go LevelDB implementation that can be cannibalized for the Table portion specifically.

`GraphDef` interpretation to actually *run* a `SavedModel` is months of work; for LOOM, the realistic goal is **weight extraction** by walking the variable list, not execution.

Keras

Legacy `.h5` is HDF5 with root attributes (`keras_version`, `backend`, `model_config`) as a JSON string of the layer tree, optional `training_config`, `/model_weights/<layer_name>/<weight_name>` datasets, `/optimizer_weights/`. HDF5 itself is a beast: superblock variants v0/v1/v2/v3, object headers v1+v2 with typed messages (dataspace, datatype, link info, layout, filter pipeline), B-trees v1 (chunked datasets, groups) and v2 (indirect attribute storage), local + global heaps, fractal heaps for dense groups, dataset storage layouts (compact/contiguous/chunked), filter pipeline (gzip/shuffle/fletcher32/szip), variable-length strings via global heap, Jenkins lookup3 checksums on headers.

The historical "HDF5 in pure Go is a non-starter" claim is no longer true as of 2025-2026.

`github.com/scigolib/hdf5` is a pure-Go HDF5 library with comprehensive read support (superblock v0/v2/v3, compact + contiguous + chunked layouts, gzip filter, fixed and variable-length strings, compound types) and write support in v0.13.5 (Feb 2026), recognized on the HDF Group's official forum. `github.com/CWBudde/go-hdf5` (also `meko-christian/go-hdf5`) is a parallel pure-Go effort. Both are **beta-quality and need validation against real-world Keras files** — large variable-length attributes (model_config JSON >64KB), dense groups with thousands of links, chunked compressed datasets with filter combinations are the edge cases that bite. `Go Packages + 4`

`.keras` (Keras 3 unified format) is a **ZIP archive containing** `config.json` (**full layer config**), `metadata.json` (`keras_version`, `date_saved`), and `model.weights.h5` — and the inner weights file **is still HDF5** as of Keras 3.4 through 3.10. The community has requested a non-HDF5 weights format but it hasn't shipped. Sharded variant: `model.save_weights(...weights.json, max_shard_size=0.25)` produces a JSON index + multiple HDF5 shards. `Keras` `Keras`

Pure-Go Keras strategy: open `.keras` as ZIP (stdlib), read `config.json`/`metadata.json` directly, extract `model.weights.h5` to a temp path or pass an `io.ReaderAt` to `scigolib/hdf5`, walk `/layers/<name>/vars/N` datasets. **Effort: 1 day for the outer ZIP layer + 1-2 weeks integrating scigolib/hdf5 against real Keras files, with budget for upstream PRs.** If `scigolib` proves unstable, the fallback is a Python-side converter to `safetensors`.

TFLite, NumPy, MLX, JAX/Flax/Orbax

TFLite is **FlatBuffers** with file identifier `"TFL3"`, schema at `tensorflow/tensorflow/blob/master/tensorflow/lite/schema/schema.fbs`. Root is `Model { version=3, operator_codes, subgraphs, buffers, signature_defs, metadata }`. **Per-op versioning** (`OperatorCode.version`) is independent of the schema version. `Buffer.data: [ubyte] (force_align:16)` enables mmap zero-copy. Quantization is in `Tensor.quantization: { scale: [float], zero_point: [long], quantized_dimension: int }` — per-tensor when `scale.length==1`, per-axis (added in schema v3) when it equals the channel count. Custom ops use `FlexBuffers` payloads in `custom_options`. INT4 (packed 2-per-byte) added recently. **Pure-Go:** `flatc --go schema.fbs` produces ~10K LoC of accessors depending only on the pure-Go `FlatBuffers` runtime. **Parsing/serialization is trivial (1-2 weeks including FlexBuffers reader); op execution for a useful 30-op subset with INT8 per-axis quant is 3-6 months.** `mattn/go-tflite` is the popular Go binding but is CGO.

NumPy `.npy`/`.npz` is gloriously simple: 6-byte magic (`\x93NUMPY`), then 1-byte major + 1-byte minor version, then a uint16 (v1.0) or uint32 (v2.0/v3.0) header length, then ASCII (v1/v2) or UTF-8 (v3) header text that is a Python dict literal (`{'descr': '<f4', 'fortran_order': False, 'shape': (3, 4)}`) padded with spaces to 64-byte alignment and terminated with `\n`, then raw bytes. **dtype grammar**: `<f4` little-endian float32, `>f8` big-endian float64, `|u1` byte-order-irrelevant uint8, etc. v2 was for huge headers (structured arrays with thousands of fields); v3 was for unicode in dtype names. `.npz` is a ZIP of `.npy` files (uncompressed by default with `np.savez`, deflate with `np.savez_compressed`). **Pure-Go**: codeberg.org/sbinet/npio (formerly github.com/sbinet/npio, now archived in favor of Codeberg) at v0.12.0 (Aug 2025) is the de-facto library — full r/w, v1/v2/v3 headers, npz subpackage, integrates with gonum/mat. **Trivial: 1 day to integrate.** `Codeberg + 3`

MLX (Apple's array framework) uses **safetensors as its primary format**, plus `.npy/.npz` and a limited GGUF reader/writer. The MLX-specific wrinkle is quantization layout: MLX stores quantized linears as packed `uint32` arrays + sidecar `<layer>.scales` (fp16/fp32) + `<layer>.biases` tensors, with `group_size ∈ {32, 64, 128}` and `bits ∈ {2, 3, 4, 6, 8}` recorded in `config.json`'s `quantization` block. MLX historically didn't set the `format` metadata key in safetensors, causing HF Transformers loading to fail (issue [ml-explore/mlx#743](#)). LoRA adapters transitioned from `adapters.npz` to `adapters.safetensors`. **Pure-Go: same difficulty as safetensors — trivial, 2-3 days for an MLX-aware safetensors loader with dequant kernels for 4/8-bit MLX packing.**

Flax legacy serialization is **msgpack** with custom ext-type encoding: each ndarray becomes an ext-type-1 object packing `(shape_tuple, dtype_name_string, raw_bytes_C_order)`, with arrays >1 GiB chunked via `_chunk_array_leaves_in_place`. **bfloat16** is preserved by literal dtype name string `"bfloat16"`. No file magic, no version field. **Pure-Go**: [vmihailenco/msgpack/v5](https://github.com/vmihailenco/msgpack/v5) supports custom ext types via `RegisterExt`; the chunk-reassembly + bfloat16 recognition adds ~300 LoC. **Trivial: 2-4 days.**

Orbax (the modern JAX checkpoint library) is **much harder**. It's a directory layout with `_CHECKPOINT_METADATA` (JSON timestamp), per-item `_METADATA` (JSON pytree structure), and a TensorStore-backed weight store. The current default stack is **Zarr v3 codecs on top of OCDBT (Optimized Checkpoint Database) on top of the filesystem kvstore**. OCDBT is Tensorstore's B+tree-on-filesystem KV store: directory contains `manifest.ocdbt` (config + version tree referencing root B+tree nodes, with uuid, compression mode, max_decoded_node_bytes, version_tree_arity_log2), plus `d/` data files (32-hex-char CAS-named) containing encoded B+tree nodes and indirect values. B+tree nodes use delta-encoded keys with CRC-32C node checksums; zstd compression on manifests/nodes (not on indirect values). Per-process subdirs `ocdbt.process_<id>/` contain independent per-writer OCDBTs that get merged. On top, each array is a Zarr v2 or v3 store — `.zarray`/`zarr.json` metadata + chunk files. **Pure-Go Zarr is itself unsolved:** `qri-io/zarr-go` is explicitly experimental ("This code is not correct and shouldn't be used"); no mature general-purpose Zarr v2/v3 Go library exists as of mid-2026, despite Python/Rust/C/C++/Java/JS/Julia/R/OCaml/Elixir implementations existing. **Effort estimate: 2-4 dev-months for read-only OCDBT + Zarr v2, plus 1-2 more for Zarr v3 with the sharding_indexed codec. Pragmatic recommendation: require users to convert Orbax checkpoints to safetensors externally; do not implement native Orbax support in LOOM v1.**

The obscure formats: priority and verdict

A consolidated assessment for the long tail:

Format	Container	Pure-Go effort	Practical value	Verdict
OpenVINO IR (<code>.xml</code>) + (<code>.bin</code>)	XML graph + raw binary weights (Medium) with offsets	Low (~500 LOC, stdlib <code>encoding/xml</code>)	Medium-high (Intel edge deployments)	Implement
Darknet (<code>.weights</code>) (+ <code>.cfg</code>)	5-int header + raw float32 stream in cfg layer order	Trivial (~150 LOC)	Medium (YOLOv1-v4 demos)	Implement
llamafile	APE executable + zip-aligned GGUF	Trivial (~50 LOC over GGUF)	High (free over GGUF)	Implement
AWQ/GPTQ packed weights	Tensors in safetensors with naming conventions	Low (a few hundred LOC of dequant)	Medium-high (LLM space)	Implement as safetensors extension
Caffe 1 (<code>.caffemodel</code> + <code>.prototxt</code>)	protobuf2 (Berkeleyvision)	Low	Low-medium (legacy zoo)	Optional
MXNet (<code>.params</code> + <code>symbol.json</code>)	Magic-prefixed binary + JSON (Wolfram Language)	Low (~300 LOC)	Low (Apache Attic since Sept 2023)	Optional
Paddle inference (<code>.pdmodel</code> + <code>.pdiparams</code>)	protobuf + raw tensor blob (Paddlepaddle)	Medium	Medium (PaddleOCR)	Optional
Caffe 2 (<code>predict_net.pb</code> + <code>init_net.pb</code>)	protobuf	Low	Very low (dead)	Skip
Paddle (<code>.pdparams</code>) (dygraph pickle)	Python pickle	Very high	Low	Skip — require conversion
Paddle Lite (<code>.nb</code>) (Naive Buffer)	Undocumented in-house (GitHub) flatbuffer-like	High	Low (mobile only)	Skip
Stable Diffusion (<code>.ckpt</code>) (raw pickle)	Python pickle with arbitrary reducers	High + dangerous	Low (safetensors won)	Skip — require conversion
TVM (<code>.tar</code>) / MLC-LLM	Native code + JSON + raw weights	Bytes readable, models are native code	None without TVM runtime	Skip

Legacy ggml (<code>.bin</code>) (pre-GGUF)	Various magics, evolved schema	Low but pointless	None	Skip — subsumed by GGUF
EXL2 / EXL3	Standard safetensors + custom in-tensor encoding	Bytes trivial, kernels hard	Low (GPU-targeted)	Skip — recommend GGUF
TensorRT (<code>.engine</code>) / (<code>.plan</code>)	Proprietary, GPU-arch-specific	Impossible	None	Skip — error out

Three notable specifics:

llamafile is just an APE executable with a `zipalign`'d GGUF appended. Two extraction strategies in Go: treat as a ZIP via `archive/zip` (it parses from the trailing EOCD, ignoring the executable prefix; the GGUF is stored uncompressed for mmap), or magic-scan for `'G', 'G', 'U', 'F'` at a 32-byte-aligned offset. ~50 LOC over a working GGUF parser.

Darknet (`.weights`) is `[major:i32, minor:i32, revision:i32, seen:i64 (if major*10+minor≥2 else i32)]` then raw float32 in cfg-file layer order: per-conv layer it's `bias[n]` then `weights[n*c*kH*kW]` (OIHW), or with batch_normalize `bias[n], bn_scales[n], bn_rolling_mean[n], bn_rolling_var[n], weights[...]`. Architecture is in the INI-style `.cfg` file.

TensorRT engines are deliberately opaque: GPU-architecture-specific (an Ampere sm_80 engine won't run on Hopper sm_90), TensorRT-minor-version-specific, undocumented binary. No public schema, no parsing path that gives meaningful information without libnvinfer. LOOM should detect and reject with a clear conversion message.

Pure-Go library ecosystem: the consolidated state of play

Beyond the format-specific libraries already cited, the foundational pure-Go pieces LOOM can rely on are mature:

- google.golang.org/protobuf — the canonical Go protobuf; reads ONNX, TF protos, CoreML, Caffe, Paddle. Use the new APIv2, not the deprecated github.com/golang/protobuf.
- github.com/google/flatbuffers Go runtime — pure-Go, ~1500 LoC; handles TFLite and [.ort](https://github.com/google/flatbuffers/blob/master/ort).
- github.com/klauspost/compress/zstd — pure-Go zstd, needed for OCDBT/Zarr if you ever pursue Orbax.
- [compress/gzip](https://github.com/klauspost/compress/gzip), [archive/zip](https://github.com/klauspost/compress/zip), [archive/tar](https://github.com/klauspost/compress/tar), [hash/crc32](https://github.com/klauspost/compress/crc32) (with Castagnoli table) — stdlib coverage is excellent.
- golang.org/x/exp/mmap — pure-Go mmap for zero-copy weight loading.

Tokenizers deserve a flag: full LLM inference requires tokenizer fidelity, and github.com/sugarme/tokenizer is pure Go (port of HF Rust tokenizers, slower with some edge-case incompatibilities), while github.com/daulet/tokenizers and github.com/gomlx/tokenizers wrap the Rust libtokenizers via CGO (best fidelity but disqualified for pure-Go). github.com/tiktoken-go/tokenizer is pure Go but OpenAI-BPE-specific.

Two genuinely unsolved pure-Go gaps remain in mid-2026: **(a) a Zarr v2/v3 implementation** (needed for Orbax), and **(b) a TF checkpoint v2 SSTable reader** (needed for any non-frozen TF SavedModel). The first is months of work to write; the second is a 2–3 week LevelDB Table port.

CoreML in one paragraph

CoreML is protobuf, defined in github.com/apple/coremltools/tree/main/mlmodel/format/*.proto. Spec versions 1 through 9 (Core ML 1 through 8/iOS 18), with v5 introducing **MIL (Model Intermediate Language)** and the [.mlpackage](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) **directory bundle** (Manifest.json + Data/com.apple.CoreML/{model.mlmodel, weights/weight.bin} — weights externalized to bypass protobuf's 2 GB limit), v7 adding 3-bit palettization and INT4 weight quant, v9 adding **stateful models with [make_state](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage)/[coreml_update_state](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) for KV caches** and multi-function MLPackages. The MIL IR is SSA-style with [Program → Function → Block → Operation](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) and compression encoded as [constexpr_*](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) ops ([constexpr_lut_to_dense](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) for palettized, [constexpr_affine_dequantize](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) for linear quant, [constexpr_blockwise_shift_scale](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) for block quant, [constexpr_sparse_to_dense](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) for pruning). **Parsing CoreML in pure Go is mechanical (1–2 weeks via [protoc-gen-go](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) plus a [.mlpackage](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) directory walker plus a [weight.bin](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage) offset reader).** **Executing CoreML in pure Go is pointless** — the entire performance proposition is the Neural Engine, models are shaped for ANE quirks (ANE-friendly Conv2D instead of Linear, layout tricks), and the MIL operator semantics are defined by the closed-source [CoreML.framework](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage). **Recommendation: read-only parsing for ecosystem completeness only; no execution, no generation.** [GitHub + 3](https://github.com/apple/coremltools/blob/main/mlmodel/format/mlpackage)

Interoperability and conversion: where weights flow

The conversion landscape divides into routine bidirectional flows around the three hub formats and lossy or one-way flows out to runtime-locked targets:

Native paths: HF Transformers ↔ safetensors (default since 2023); MLX ↔ safetensors; PyTorch ↔ ONNX via `torch.onnx.export(dynamo=True)` (default in torch ≥2.9, with common failure modes around control flow needing `torch.cond()`, RNN graph breaks in TorchDynamo, dynamic shapes needing `dynamic_shapes` dict, custom ops needing symbolic registration); HF ↔ GGUF via llama.cpp's `convert_hf_to_gguf.py` (lossless for F16/BF16, lossy for `--outtype q8_0/q4_k_m/tq1_0/tq2_0`); TF ↔ ONNX via `tf2onnx`; Keras ↔ TFLite via `tf.lite.TFLiteConverter.from_keras_model(...)`.

One-way to runtime-locked: ONNX → TensorRT via `trtexec` or `polygraphy`; ONNX → OpenVINO IR via `mo` (Model Optimizer); ONNX → CoreML deprecated, current path is PyTorch → CoreML via `coremltools.convert` with `torch.jit.trace` or `torch.export`; PyTorch → ExecuTorch `.pte` via `torch.export`. (Intel Community)

Lossy or impractical: GGUF → safetensors loses the K-quant/IQ-quant block structure (dequantized weights are recoverable, original packing is not); GPTQ ↔ AWQ ↔ GGUF Q4_K are not 1:1 — each uses different group sizes and packing layouts, requiring dequant + requant; ONNX → PyTorch via `onnx2torch/onnx2pytorch` loses framework-native ops. CoreML, TensorRT engines, OpenVINO IR have no useful reverse paths.

BitNet b1.58: the LOOM-specific story

BitNet b1.58 models exist in three coexisting distribution forms, all with implications for LOOM:

1. **Safetensors BF16/F32 master weights** (e.g. `microsoft/bitnet-b1.58-2B-4T-bf16`, `1bitLLM/bitnet_b1_58-large`): ternary $\{-1, 0, +1\}$ values stored in full-precision tensors, 5–10× larger than packed ternary. LoRA-fine-tunable. Architecture deviates from LLaMA: extra `ffn_sub_norm` and `attn_sub_norm` RMSNorm tensors. **LOOM reads these today via safetensors.** (arxiv) (arxiv)
2. **Safetensors packed ternary** (e.g. `microsoft/bitnet-b1.58-2B-4T` main repo): Microsoft's custom byte-packed ternary format, runtime-locked to the `bitnet.cpp` reference implementation.
3. **GGUF with TQ1_0/TQ2_0/i2_s** (e.g. `microsoft/bitnet-b1.58-2B-4T-gguf` with `ggml-model-i2_s.gguf` + `t11` for ARM CPUs): the ternary types added in llama.cpp PR #8151. Note that current BitNet GGUFs are often larger than they need to be because `token_embd.weight` and `output.weight` stay in F16 or Q4_K/Q6_K rather than ternary.

The high-leverage LOOM move: GGUF reader + TQ2_0 + TQ1_0 dequant kernels in pure Go — Microsoft's reference BitNet GGUFs become directly consumable, and no existing pure-Go library does this. TQ2_0 is half a day's work (2-bit lookup with one fp16 scale per 256-element block); TQ1_0 is 1-2 days (the base-3 multiply trick). Combined with LOOM's existing ternary execution path, this is the most distinctive ecosystem position LOOM can occupy.

Recommended LOOM integration roadmap

Sequenced for maximum ecosystem coverage per engineer-week, with reasoning:

Phase 1 (≤1 week): finish the safetensors story. Complete writer support, sharded index reader (`(model.safetensors.index.json)`), and FP8/FP4 dtype handling. Validate against `(nlpodyssey/safetensors)` for byte-level parity. This buys the entire HuggingFace Transformers ecosystem since 2023 and all MLX models.

Phase 2 (2–3 weeks): GGUF reader with the strategic quant subset. Header + KV metadata (including recursive ARRAY values) + tensor info + alignment + mmap. Reference `(gpustack/gguf-parser-go)` for the parser layer. Then dequant kernels: F32/F16/BF16 (type-punning), then **TQ2_0 then TQ1_0** (the BitNet differentiator), then Q8_0, Q4_0, Q4_K, Q6_K. Stop here for v1 unless a specific model needs more. The architecture metadata registry (llama, qwen2/3, mistral, phi3, gemma, bitnet) is a separate 2–3 days per family.

Phase 3 (1–2 weeks): GGUF writer + llamafile + Darknet + NumPy. GGUF writer enables producing files from LOOM. Llamafile extraction is 50 LoC over GGUF. Darknet `(.weights)` is 150 LoC and unlocks YOLOv1–v4 demos. `(.npy)/(.npz)` via `(codeberg.org/sbinet/npzio)` is one-day integration. Flax legacy msgpack adds another 2–4 days via `(vmihailenco/msgpack)`.

Phase 4 (2–4 weeks): ONNX protobuf parsing with a curated executor. Generate Go bindings from `(onnx.proto)` (vendor `(onnx-ml.proto)` extension at IR 13 / opset 25). Implement external-data resolver with mmap, sub-byte unpacking helpers, FP8/FP4 → fp32 conversion. Then execution for ~30 LLM-shaped ops: Attention-23 + GroupQueryAttention (com.microsoft) + RotaryEmbedding + RMSNormalization, LayerNormalization-17, MatMul + **MatMulNBits (the 4-bit weight-only workhorse)**, Gather, Add/Mul/Softmax/SiLU/GELU, Reshape/Transpose/Concat, Q/DQ with INT4. Reject opset combinations outside this subset with a clear error. Parse-only `(.ort)` via `(flatc --go ort.fbs)` adds one extra week.

Phase 5 (opportunistic): PyTorch state_dict via gopickle. Integrate `nlpodyssey/gopickle` with a hard-coded allowlist mirroring `_weights_only_unpickler.py`. Document clearly that TorchScript, `.pt2`, and arbitrary pickled `nn.Module` subclasses are not supported and that users should convert to safetensors via `transformers` first. Add defensive limits (opcode count, stack depth, memo size, timeout).

Phase 6 (deferred / contingent): Keras `.keras` + `.h5` via scigolib/hdf5. Re-evaluate after scigolib hits a v1.0 release. The `.keras` outer ZIP layer is one day; the inner HDF5 traversal needs real-Keras-file validation and likely upstream PRs to scigolib. Plan a Python-side fallback converter (`h5py` → safetensors) for files that fail.

Phase 7 (defer indefinitely): TF SavedModel. Frozen `.pb` graphs (TF1, weights as Const NodeDefs) are a 2-day protobuf-only path worth shipping early. SavedModel checkpoint v2 reading requires the **3-week LevelDB SSTable port** — only worth it if there's clear user demand. Execution is months on top.

Skip in v1: TVM/MLC-LLM (native code), TensorRT engines (proprietary), CoreML execution (Apple-runtime-locked), arbitrary pickled PyTorch models, Stable Diffusion raw `.ckpt`, Paddle dygraph `.pdparams`, Paddle Lite `.nb`, EXL2 kernels, Orbax (multi-month Zarr+OCDBT effort). Document clear conversion paths for each.

Where the report shifts the priors

Three findings should update Sam's mental model:

HDF5 in pure Go is no longer a blocker, but it is a known-new risk. `scigolib/hdf5` (v0.13.5 as of Feb 2026, recognized on the HDF Group's forum) genuinely covers Keras's HDF5 needs in pure Go — superblock v0/v2/v3, contiguous + chunked layouts, gzip filter, variable-length strings. The risk shifts from "impossible" to "betting on a v0.x library against real-world Keras files." Plan a Python-converter fallback.

TF checkpoint v2 SSTable is the one genuine remaining blocker that's tractable but unaddressed. Every existing Go TF binding uses CGO. A 2-3 week LevelDB Table port (cribbing from `syndtr/goleveldb`) plus 2 days of BundleEntryProto would unlock all TF SavedModel weight extraction. Worth it only if user demand emerges; not a v1 priority.

The BitNet TQ1_0/TQ2_0 pure-Go dequant kernels are LOOM's unique strategic opportunity.

No pure-Go library today dequantizes any GGUF K-quant or I-quant or TQ-quant. The TQ variants — being structurally simple (one fp16 scale + base-3 or base-4 packing, no codebook tables) and directly aligned with LOOM's existing BitNet b1.58 native execution path — are the highest-leverage hard target. Two days for TQ2_0, plus one or two more for TQ1_0, plus a working GGUF parser, makes LOOM the only pure-Go engine that can load Microsoft's reference BitNet GGUFs.